

Automated Recursive HTML Scraper for Clean Exports

GAGAN K M



Why Automate HTML Content Collection

Reduce time, errors, duplicates, and scale reliably



Time cost of manual collection

Copy and paste across many pages takes hours; automation reduces to minutes.



Why NO OCR Method

Manual capture misses formatting, loses metadata, and Cause lot of Time Delay



Duplicate and repetitive text

Manual methods may add repeated paragraphs; automation enables systematic and clean Contents.

End-to-End Flow: URL to Downloadable .docx

Linear process showing recursive scraping, and export

■ User enters start URL

- Begin with the user-provided URL as the crawl seed

■ Crawler starts (queue seeded)

- Seed crawl queue and initialize crawler components

■ Recursive drill: discover links max depth

- Recursively follow discovered links while respecting depth limits

■ Parse page HTML and extract visible paragraphs

- Render or parse HTML and extract **visible paragraph text** from each page
- Detect and remove repetitive or duplicate paragraphs via hashing

■ Assemble content and add metadata

- Combine unique content dropping duplicates paragraphs.

■ Save .docx and prompt user to download

- Export assembled document to .docx and present download prompt

Input Handling: URL Validation and Queue Initialization

Prepare and enqueue seed URLs

01 Validation sequence



- Protocol check: accept http and https only
- Parse and canonicalize URLs to avoid duplicates

02 Seed queue and visited set



- Store normalized form for duplicate detection
- Maintain visited set to prevent repeats
- Pop next URL respecting depth limits

03 User controls



- Start and stop crawl on demand
- Set maximum crawl depth

Step 3: Parsing HTML to Extract Visible Paragraph Text

Clean, select, and capture readable content for storage

01

Parsing tasks

Remove script and style, strip HTML tags, collapse whitespace, decode entities.

02

Dynamic content handling

Pages requiring JavaScript need a headless browser like Playwright or Selenium, or use APIs.

03

Example tools

BeautifulSoup and lxml for static pages; Playwright or Selenium for rendered content.

Duplicate Detection and Content Cleaning Strategies

Practical methods to find and remove redundant paragraphs

Exact and deterministic

1. Hashing: compute SHA-1 or MD5 of normalized paragraph text
2. Exact hash match means duplicate; fast and deterministic

Near-duplicate and fuzzy detection

1. Fuzzy matching: cosine similarity or Levenshtein; threshold around 85% similarity
2. Action: similarity $\geq 85\%$ $\rightarrow$ review or auto-merge; discard boilerplate templates

Storage and Export: Assemble a Clean .docx

Order, format, metadata, export, and timestamped filename

01



Assemble content

Order paragraphs by source priority or crawl order; include source URL and capture timestamp per section.

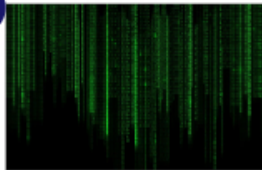
02



Formatting

Use headings for source pages, paragraph spacing, and simple styles for readability.

03



Metadata

Include header with start URL, crawl date 2025-12-24, depth, and item count.

04



Save and prompt

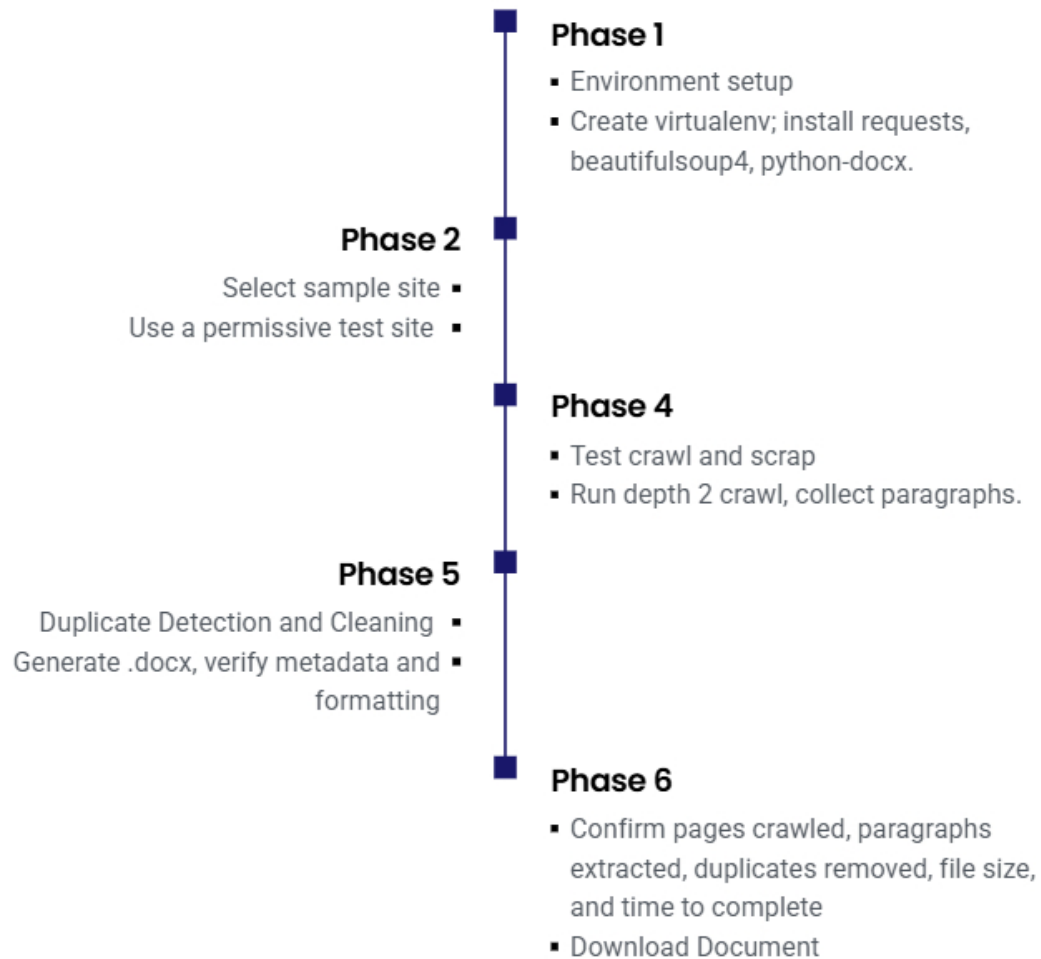
Export using python-docx to .docx; then prompt user to download or save with timestamp.

05



File naming convention

Use pattern `scrape__YYYYMMDD_HHMM.docx`, e.g., `scrape_example.com_20251224_0623.docx`.



Next Steps: Demo Checklist and Minimal Roadmap